Computer Networks Laboratory

Comprehensive Lab Guide

Labs 01 – 07 | Cisco Packet Tracer & Wireshark

CL3001 – Spring 2026

FAST-NUCES, Karachi

Topics Covered

Networking Commands • ARP & Ethernet • Python Sockets
Cisco Packet Tracer Topologies • Telnet & SSH • VLANs
HTTP/HTTPS • DNS • DHCP • FTP • SMTP • ACLs
Wireshark Packet Analysis

How to Use This Guide

Each lab chapter is structured in two parts:

- Theory & Concepts – explains WHY each device, IP address, service, or protocol exists and what problem it solves.
- Step-by-Step Lab Tasks – walks through exactly what to configure in Cisco Packet Tracer or Wireshark, with commands and expected outputs.

TIP	Read the Theory section before starting the lab. Understanding WHY helps you remember HOW.

NOTE	Screenshots are required throughout. Capture every command output and Packet Tracer simulation before moving on.

Lab 01 – Essential Networking Commands

Theory: Why Networking Commands Matter

Before configuring any network topology, a network engineer must be able to inspect and diagnose the local machine and its connections. The operating system exposes this through a set of built-in command-line tools. Each tool maps directly to one or more layers of the OSI model.

The OSI Model Quick Reference

Layer	Name	Purpose	Relevant Command
7	Application	User-facing protocols (HTTP, DNS)	curl, nslookup
4	Transport	Port-to-port delivery (TCP/UDP)	netstat
3	Network	IP routing across networks	ping, tracert, ipconfig
2	Data Link	MAC addressing, same-LAN delivery	arp -a
1	Physical	Cables, signals	Cable selection

Why Does Your Computer Have an IP Address?

An IP address is the unique identifier that allows your machine to send and receive packets across a network. Without it, no other device could direct traffic specifically to you. The subnet mask tells your computer which portion of the address identifies the network versus the individual host. The default gateway is the router – the door through which your traffic exits to the wider internet.

Why Does Your Computer Also Have a MAC Address?

MAC (Media Access Control) addresses operate at Layer 2. While IP addresses route traffic between networks, MAC addresses deliver a frame within a single LAN segment. The ARP (Address Resolution Protocol) bridges the two: when your computer knows the destination IP but not the MAC, it broadcasts an ARP request, and the target replies with its MAC. This mapping is cached in the ARP table.

WHY	Without ARP, even a perfectly configured IP network would fail to deliver frames within the same LAN. ARP is invisible but critical.

Lab Tasks with Answers

Task 1 – Find Your IP Address

Field	Details
Command (Windows)	ipconfig
Command (Linux/Mac)	ifconfig *or* ip addr
What to look for	IPv4 Address line under your active adapter
Why this command	ipconfig reads the network stack configuration from the OS kernel and displays it in readable form

```
C:\> ipconfig

Windows IP Configuration
Ethernet adapter Ethernet:
  IPv4 Address . . . . . : 192.168.1.105
  Subnet Mask  . . . . . : 255.255.255.0
  Default Gateway  . . . : 192.168.1.1
```

Task 2 – Full Network Info (IP, MAC, Gateway, DHCP)

Use ipconfig /all to see the complete picture including MAC address and whether DHCP is assigning your IP automatically.

```
C:\> ipconfig /all

  Physical Address . . . : A4-91-B1-3C-22-DD   <- MAC address
  DHCP Enabled . . . . . : Yes                 <- DHCP is on
  IPv4 Address . . . . . : 192.168.1.105
  Default Gateway  . . . : 192.168.1.1
```

WHY DHCP?	DHCP (Dynamic Host Configuration Protocol) eliminates manual IP management. A DHCP server assigns an IP, subnet mask, gateway, and DNS server automatically when a device connects. Without DHCP, every device on a large network would need to be manually configured.

Task 3 – Display the Hostname

```
C:\> hostname

DESKTOP-K7F2LMN
```

The hostname is the human-readable name for your machine on the network. DNS maps this name to an IP address so other computers can reach your machine by name instead of number.

Task 4 – Check IP Connectivity with Ping

Ping sends ICMP Echo Request packets to a target. If the target is alive and reachable, it sends back ICMP Echo Reply packets. Four packets are sent by default.

```
C:\> ping 192.168.1.1

Reply from 192.168.1.1: bytes=32 time=1ms TTL=64
Reply from 192.168.1.1: bytes=32 time=1ms TTL=64

C:\> ping computername   <- ping by name (requires DNS or hosts file)
```

Common reasons ping fails:

- The target machine has a firewall blocking ICMP.
- The IP address is incorrect, or the device is powered off.
- There is no route between the source and destination networks.
- A switch or router in the path is misconfigured.

Task 5 – View Open Ports (netstat)

netstat (Network Statistics) shows all active TCP/UDP connections and the ports your applications are listening on. This is how you verify that a web server, for example, is listening on port 80.

```
C:\> netstat -an

Proto  Local Address        Foreign Address     State
TCP    0.0.0.0:80           0.0.0.0:0           LISTENING <- web server
TCP    192.168.1.105:50124  44.228.249.3:443    ESTABLISHED <- HTTPS
```

Task 6 – Trace the Route to google.com

tracert (traceroute on Linux) sends packets with increasing TTL (Time To Live) values. Each router that decrements TTL to zero replies with a 'TTL Exceeded' message, revealing itself. This maps the entire path from source to destination hop by hop.

```
C:\> tracert www.google.com

  1   1 ms    192.168.1.1       <- your home router
  2   8 ms    10.0.0.1          <- ISP gateway
  3   22 ms   172.217.163.46    <- Google edge server
  ...
Trace complete.
```

Each numbered line is a 'hop'. The number of hops tells you how many routers traffic passes through. High hop counts or timeouts indicate congestion or routing issues.

Task 7 – Ping Works by IP, Fails by Name

If ping 192.168.0.2 succeeds but ping blue-machine fails, the IP layer is fine, but name resolution is broken. This means DNS (or the local hosts file) does not contain an entry mapping the name 'blue machine' to that IP address. Solution: add an entry to the hosts file (C:\Windows\System32\drivers\etc\hosts) or configure a DNS server.

Tasks 8 & 9 – UTP Cables

In a home installation, straight-through Cat5e/Cat6 cables are standard for connecting PCs and laptops to wall sockets, patch panels, and switches.

Connection Type	Cable Required	Why
PC to Switch	Straight-through	Different device types – TX on one side connects to RX on the other
Switch to Switch	Crossover (or auto-MDI-X)	Same device types – need to cross TX/RX pairs manually
Router to Router	Crossover	Same device types
PC to PC (direct)	Crossover	Same device types
PC to Router (console)	Rollover / Console cable	Serial management access

NOTE	Modern switches and routers support Auto-MDI-X, which automatically detects and corrects the cable type, so crossover cables are rarely required on new equipment.

Lab 02 – ARP, Switches, Hubs, and Bridges

Theory: The Data Link Layer (Layer 2) Devices

This lab focuses on how devices within the same LAN communicate. It introduces three key devices – Hub, Bridge, and Switch – each representing a generation of network technology. Understanding why switches replaced hubs, and why bridges divided broadcast domains, forms the foundation for all later network design.

How Devices Find Each Other: ARP

When Computer A wants to send data to Computer B on the same LAN, it knows B's IP address (from DNS or manual configuration) but does not know B's MAC address. The process works as follows:

1. **A** checks its ARP cache (arp -a) for an existing IP-to-MAC mapping.
2. If none exists, **A** broadcasts an ARP Request to every device on the LAN: '*Who has IP 192.168.1.20? Tell 192.168.1.10*'.
3. Only the device with IP 192.168.1.20 replies with an ARP Reply: '*I am MAC AA:BB:CC:DD:EE:FF*'.
4. **A** stores this mapping in its ARP cache and sends the data frame.

In Cisco Packet Tracer, you can observe ARP by switching to Simulation Mode, filtering for ARP packets, and watching the broadcast propagate to every device before the reply returns.

WHY ARP?	ARP is the bridge between the Layer 3 IP world and the Layer 2 MAC world. Without it, IP routing between hosts on the same subnet would be impossible regardless of how IP addresses are configured.

The ARP Packet Header Format

When Packet Tracer displays an ARP packet in simulation mode, you will see these fields:

Field	Size	Value / Meaning
Hardware Type	2 bytes	1 = Ethernet
Protocol Type	2 bytes	0x0800 = IPv4
Hardware Addr Length	1 byte	6 = MAC address length

Protocol Addr Length	1 byte	4 = IPv4 address length
Operation	2 bytes	1 = Request, 2 = Reply
Sender MAC	6 bytes	MAC of the device sending the ARP
Sender IP	4 bytes	IP of the device sending the ARP
Target MAC	6 bytes	FF:FF:FF:FF:FF:FF for request (broadcast)
Target IP	4 bytes	The IP being queried

Hub vs. Switch – A Critical Distinction

This distinction is one of the most tested concepts in networking. Both devices connect multiple hosts in a LAN, but they behave very differently.

Property	Hub (Layer 1)	Switch (Layer 2)
Collision Domains	One for all ports	One per port
Broadcast Domains	One for all ports	One per VLAN (one if no VLANs)
How it works	Repeats every bit to ALL ports	Learns MAC table, forwards only to correct port
Bandwidth	Shared among all devices	Dedicated per port
Security	Every device sees all traffic	Only destination receives unicast traffic
Performance	Degrades with more devices	Scales well
Intelligence	None (dumb repeater)	Maintains MAC address table

WHY SWITCHES?	A hub creates a shared collision domain. With 8 PCs on a hub, only one can transmit at a time or frames collide. A switch eliminates collisions by giving each port its own collision domain and forwarding only to the intended port. This is why hubs are obsolete in modern networks.

What is a Bridge?

A bridge connects two or more LAN segments and filters traffic between them based on MAC addresses. Unlike a hub (which blindly repeats everything), a bridge learns which MAC addresses are on which side and only forwards frames that need to cross. This creates separate collision domains on each side while keeping them in the same broadcast domain.

In Packet Tracer Task 4, the topology uses two Hubs connected by a Bridge. Each hub forms one broadcast segment. The bridge separates them into two collision domains, but both sides still share one broadcast domain (all 8 PCs receive broadcasts). This mirrors how early corporate networks were segmented before switches became affordable.

UTP Cable Types for Specific Connections

The figure in the lab shows a PC connected to a switch console port with a Rollover cable, while other connections use Fast Ethernet links. Here is the mapping:

Segment	Connection	Cable Type	Reason
1	PC to Switch console port	Rollover (Console)	Serial management – not a data connection
2	Switch to Switch	Crossover Cat5e/6	Same device type – TX/RX must be swapped
3	Switch to Router	Straight-through Cat5e/6	Different device types – standard wiring

Lab Tasks – Cisco Packet Tracer Setup

Task 1 – ARP Topology and Connectivity

5. Open Cisco Packet Tracer. Add 2 switches (2960), 1 server, and 4 PCs matching the given topology.
6. Connect PC0 and PC1 to Switch0. Connect PC2 and PC3 to Switch1. Connect Server0 to Switch0. Connect the two switches with a crossover cable (Packet Tracer auto-selects for you).
7. Assign static IPs to all devices in the 192.168.1.0/24 range (e.g., PC0=.10, PC1=.11, Server=.1).
8. Open a PC's Command Prompt (Desktop > Command Prompt) and run, ping 192.168.1.1
9. Switch to Simulation Mode. Filter for ARP packets only. Click Play. Observe the ARP broadcast propagate and the unicast reply return.

10. Click on the ARP packet envelope and examine the PDU formats tab to see the header fields described above.

Task 4 – Hub and Bridge Network (8 PCs, 2 Broadcast Domains)

11. Place 2 Hubs and 1 Bridge in the workspace.
12. Connect PC0, PC1, PC2, PC3, Server1 to Hub0. Connect PC4, PC5, PC6, PC7, Server2 to Hub1.
13. Connect Hub0 to Bridge0 port 0 and Hub1 to Bridge0 port 1 using straight-through cables.
14. Assign IPs: Hub0 side uses 192.168.1.0/24, Hub1 side uses 192.168.2.0/24.
15. Send a ping from PC0 (192.168.1.10) to PC3 (192.168.1.13) – same segment, no bridge traversal needed.
16. Send a ping from PC0 to PC4 (192.168.2.10) – crosses the bridge. Observe the bridge learning MAC addresses and only forwarding relevant frames.

OBSERVE	In Simulation Mode, watch how a Hub floods every frame to all ports, while the Bridge only forwards a frame to the correct side after learning the MACs. This visually demonstrates why bridges improve efficiency.

Lab 03 – Python Socket Programming (Client-Server Chat)

Theory: Sockets and TCP/IP

A socket is the software abstraction that allows a program to send and receive data over a network. Think of a socket as one end of a telephone call – you open it, connect to someone, talk, and close it. The operating system handles the actual TCP/IP mechanics underneath.

Why TCP for a Chat Application?

TCP (Transmission Control Protocol) guarantees that data arrives in order, without duplication, and without loss. For a chat application these guarantees matter: you do not want messages arriving out of sequence or silently dropped. UDP would be faster but unreliable – suitable for video streaming, not messaging.

Ports and IP Addresses in Sockets

A server must bind to a specific IP address and port number. The port number identifies the specific service running on that machine (e.g., port 80 = HTTP, port 443 = HTTPS, port 22 = SSH). In our chat server we choose a high-numbered port (e.g., 12345) that is unlikely to be in use by another service. Clients connect to the server's IP and that port.

WHY 127.0.0.1?	127.0.0.1 (localhost) is the loopback address – it routes traffic back to your own machine without involving any network hardware. Use this to test client and server on the same computer during development.

Threading: Why It Is Needed for Multiple Clients

A basic server handles one client at a time: accept a connection, receive data, send reply, close. If a second client connects while the first is still talking, it waits. For a chat room, this is unacceptable. Threading solves this by spawning a new thread for each accepted connection, allowing the main server loop to immediately accept the next client while each thread handles its own client independently.

Task 1 – Simple Client-Server Chat with File Transfer

Server Code (server.py)

```
import socket
import os

HOST = '127.0.0.1'
PORT = 12345
BUFFER = 4096

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))    # Attach to IP and port
server.listen(1)             # Allow 1 pending connection
print('[SERVER] Waiting for connection...')

conn, addr = server.accept() # Block until a client connects
print(f'[SERVER] Connected: {addr}')

while True:
    data = conn.recv(BUFFER)       # Receive up to 4096 bytes
    if not data: break
    msg = data.decode()
    if msg.startswith('FILE:'):    # File transfer protocol
        fname = msg.split(':')[1]
        size  = int(conn.recv(BUFFER).decode())
        file_data = conn.recv(size)
        with open('received_' + fname, 'wb') as f:
            f.write(file_data)
        print(f'[SERVER] File received: {fname}')
        conn.send(b'[FILE RECEIVED]')
    else:
        print(f'[CLIENT]: {msg}')
        conn.send(b'[OK]')

conn.close()
server.close()
```

Client Code (client.py)

```
import socket
import os
```

```
HOST = '127.0.0.1'
PORT = 12345

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))   # Connect to server

while True:
    msg = input('You: ')
    if msg.lower() == 'quit': break

    if msg.startswith('/send '):  # File send command
        path = msg.split(' ', 1)[1]
        fname = os.path.basename(path)
        with open(path, 'rb') as f:
            data = f.read()
        client.send(f'FILE:{fname}'.encode())
        client.send(str(len(data)).encode())
        client.send(data)
    else:
        client.send(msg.encode())

    reply = client.recv(1024).decode()
    print(f'Server: {reply}')

client.close()
```

Task 2 – Multi-Client Chat Room with Threading

Why This Architecture Works

The server maintains a list of all active client connections. When one client sends a message, the server broadcasts it to every other client in the list. Each client connection is managed by its own thread so no single slow client blocks the others.

```
import socket, threading

HOST, PORT = '0.0.0.0', 12345    # '0.0.0.0' = accept on all interfaces
clients = []                     # Shared list of active connections
lock = threading.Lock()          # Protect shared list from race conditions

def broadcast(msg, sender=None):
    with lock:
        for c in clients:
```

```
        if c != sender:      # Don't echo back to sender
            try: c.send(msg)
            except: clients.remove(c)

def handle_client(conn, addr):
    print(f'[+] {addr} connected')
    with lock: clients.append(conn)
    while True:
        try:
            data = conn.recv(4096)
            if not data: break
            broadcast(data, sender=conn)
        except: break
    with lock: clients.remove(conn)
    conn.close()
    print(f'[-] {addr} disconnected')

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(10)
print('[SERVER] Chat room open')

while True:
    conn, addr = server.accept()
    t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
    t.start()
```

Task 3 – Server-Side Validation

Why Validate on the Server?

Client-side validation can always be bypassed by a malicious user who sends raw socket data. Server-side validation is the authoritative gate. The server must verify both file type (by checking the extension) and message content (by scanning for banned words) before relaying anything to other clients.

```
ALLOWED_EXTENSIONS = {'.txt', '.jpg', '.jpeg', '.pdf', '.png'}
BANNED_WORDS = ['badword1', 'badword2', 'spam']  # expand as needed

def validate_message(text):
    lower = text.lower()
    for word in BANNED_WORDS:
```

```
        if word in lower:
            return False, f'Message rejected: contains banned word "{word}"'
    return True, None

def validate_file(filename):
    _, ext = os.path.splitext(filename)    # os.path.splitext extracts extension
    if ext.lower() not in ALLOWED_EXTENSIONS:
        return False, f'File type "{ext}" is not allowed.'
    return True, None

# In handle_client, before broadcasting:
if msg.startswith('FILE:'):
    fname = msg.split(':')[1]
    ok, reason = validate_file(fname)
    if not ok:
        conn.send(f'[SERVER] {reason}'.encode())
        continue   # Do not relay the file
else:
    ok, reason = validate_message(msg)
    if not ok:
        conn.send(f'[SERVER] {reason}'.encode())
        continue
```

Lab 04 – Cisco Packet Tracer: Telnet & SSH

Theory: Remote Management of Network Devices

Why Do Switches Need IP Addresses?

A switch operates at Layer 2 and does not need an IP address to forward frames. However, for remote management (logging in from a PC to configure the switch via the network), the switch must have a management IP address assigned to its VLAN interface. Without it, you can only configure the switch by physically connecting a console cable.

Telnet vs. SSH – Why SSH Matters

Property	Telnet	SSH (Secure Shell)
Encryption	None – all data including passwords sent in plaintext	Full encryption using public-key cryptography
Port	23	22
Security	Anyone on the LAN can intercept credentials	Traffic is unreadable even if intercepted
Use Today	Lab/testing only – never on production	Standard for all remote device management
Cisco Setup	transport input telnet	crypto key generate rsa + transport input ssh

WHY SSH?	In a real organization, Telnet is a security liability. A packet capture on the same network reveals your switch admin password in cleartext. SSH encrypts the session so only the authorized administrator can read the commands. Always use SSH in production.

IP Address Assignment Scheme

The lab uses a roll-number-based IP scheme: XX.XX.YY.0 where XX.XX comes from your roll number (e.g., roll 3879 → 38.79) and YY increments for each network segment (1, 2, 3…). This ensures every student has a unique address space, preventing confusion when comparing screenshots.

Lab Exercise I – Multi-Switch Topology

Step-by-Step Configuration

17. Build the topology: PC0 and PC1 on Switch1; Server-PT, PC2 on Switch2; PC3, Laptop0 on Switch3; PC4 on Switch3. Interconnect the switches as shown (Switch1–Switch2–Switch3 with crossover connections).
18. Assign IPs to PCs. Following roll number 3879 example: Network 1 = 38.79.1.0/24 (Switch1 side), Network 2 = 38.79.2.0/24, Network 3 = 38.79.3.0/24.
19. Assign management IPs to switches. On Switch1 for example:

```
Switch> enable
Switch# configure terminal
Switch(config)# interface vlan 1
Switch(config-if)# ip address 38.79.1.100 255.255.255.0
Switch(config-if)# no shutdown
Switch(config-if)# exit
Switch(config)# ip default-gateway 38.79.1.1
Switch(config)# line vty 0 4
Switch(config-line)# password cisco123
Switch(config-line)# login
Switch(config-line)# transport input telnet
Switch(config-line)# end
Switch# write memory
```

20. Test ping from any PC to the server to verify Layer 3 connectivity.
21. Verify Telnet: Open PC's Command Prompt and type: telnet 38.79.2.100 (Switch2's management IP). Enter the VTY password when prompted.
22. Change Switch2's IP from PC2's command prompt via Telnet – type the new IP command sequence as shown above.

Lab Exercise II – Router with SSH

Router Interface Configuration (CLI)

Unlike switches, routers need IPs assigned directly to their physical interfaces (FastEthernet ports), not to a VLAN interface. Each router interface connects to a different network subnet.

```
Router> enable
Router# configure terminal
Router(config)# interface fastEthernet 0/0
Router(config-if)# ip address 38.79.1.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
```

```
Router(config)# interface fastEthernet 0/1
Router(config-if)# ip address 38.80.2.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
```

Enabling SSH on the Router

```
Router(config)# hostname Router1              # Hostname required for SSH key generation
Router1(config)# ip domain-name lab.local  # Domain required for RSA key
Router1(config)# crypto key generate rsa      # Generate the RSA keypair
  How many bits in the modulus [512]: 1024
Router1(config)# username admin privilege 15 password admin@123
Router1(config)# ip ssh version 2
Router1(config)# line vty 0 4
Router1(config-line)# transport input ssh
Router1(config-line)# login local             # Use local username database
Router1(config-line)# exit
```

From Laptop0, open Desktop > Terminal or Command Prompt and type: ssh -l admin 38.79.1.1

WHY 1024-BIT RSA?	RSA keys are used to encrypt the SSH session. 1024-bit is the minimum Packet Tracer supports. In production networks, 2048-bit or 4096-bit keys provide much stronger security.

Lab 05 – HTTP/HTTPS, DNS, and Wireshark

Theory: The Application Layer Protocols

What is HTTP and Why Was It Invented?

HTTP (Hyper Text Transfer Protocol) is the protocol that powers the World Wide Web. When your browser loads a page, it sends an HTTP GET request to the server. The server replies with an HTTP response containing the HTML, images, and other content. HTTP is stateless – each request-response pair is independent; the server remembers nothing about previous requests by default.

HTTP uses TCP port 80 and operates at the Application Layer (Layer 7). Because TCP is used underneath, HTTP benefits from reliable, ordered delivery without having to implement its own retransmission logic.

Why HTTPS Replaced HTTP

HTTP sends everything in plaintext. If you log in to a banking website over plain HTTP, anyone on the same Wi-Fi network can capture your username and password using a tool like Wireshark. HTTPS solves this by wrapping HTTP inside TLS (Transport Layer Security), creating an encrypted tunnel. Only the server with the matching private key can decrypt the traffic.

Feature	HTTP	HTTPS
Port	80	443
Encryption	None – all data visible to anyone on the path	AES/ChaCha20 – only endpoints can read it
Certificate	Not required	Required – proves server identity
URL Prefix	http://	https://
Layer	Application (Layer 7)	Application (7) + Transport Security (TLS)
Suitable for	Static public pages only	Anything with login, payments, or private data
Wireshark view	Full request/response readable	Encrypted blob – content unreadable

WHY CERTIFICATES?	An HTTPS certificate (SSL/TLS certificate) serves two purposes: it enables encryption AND it proves the server is who it claims to be. Without certificate verification, an attacker could set up a fake server with encryption but steal your credentials anyway. Certificate Authorities (CAs) act as trusted third parties that vouch for server identities.

HTTP Status Codes – What They Mean

Every HTTP response contains a 3-digit status code. These codes fall into five classes:

Code Range	Category	Common Examples
2xx	Success	200 OK – request succeeded
3xx	Redirection	301 Moved Permanently – URL has changed
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 408 Timeout
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

What is DNS and Why We Need It

The Domain Name System (DNS) is the internet's phone book. Humans remember names like google.com; computers route traffic using IP addresses like 142.250.184.78. DNS translates between the two. Without DNS, you would need to memorize IP addresses for every website you visit.

DNS is hierarchical and distributed. At the top are root servers, below them are Top-Level Domain (TLD) servers (.com, .org, .pk), below them are authoritative name servers for each domain. Your local DNS resolver queries this hierarchy to resolve any name.

DNS Record Types You Need to Know

Record Type	Purpose	Example
A Record	Maps a domain name to an IPv4 address	fast-ai → 35.0.0.1
AAAA Record	Maps a domain name to an IPv6 address	fast-ai → 2001:db8::1

CNAME Record	Alias – maps one name to another name	fast → fast-ai (fast resolves via fast-ai's A record)
MX Record	Identifies mail servers for a domain	mail.example.com
NS Record	Identifies authoritative name servers for a zone	ns1.example.com
SOA Record	Start of Authority – zone metadata and serial number	Serial, refresh, retry timers

WHY CNAME?	CNAMEs let you point multiple hostnames at the same server without duplicating A records. If the server's IP changes, you only update one A record and all CNAME aliases automatically resolve correctly.

Lab Tasks – HTTP/HTTPS in Cisco Packet Tracer

Task 1a – HTTP Implementation

23. Build the topology: Router1 (1841) with Fa0/0 on 192.168.1.0/24 and Fa0/1 on 192.168.2.0/24. Connect Switch to Fa0/3. Connect PC0 (192.168.1.2), PC1 (192.168.1.3) to the Switch. Connect Server0 (192.168.2.2) directly to Router Fa0/1.
24. Why these IPs? The router separates two different subnets. PC0 and PC1 are in the 192.168.1.x subnet (the client/lab subnet). Server0 is in the 192.168.2.x subnet (the server room subnet). The router forwards packets between them.
25. Click Server0 > Services tab > HTTP. Confirm HTTP and HTTPS are both ON. You will see default HTML files listed (index.html, helloworld.html, etc.).
26. Click PC0 > Desktop > Web Browser. Type: http://192.168.2.2 and click Go. The Cisco Packet Tracer default web page should load.
27. Switch to Simulation Mode. Edit Filters to show only HTTP. Click Capture/Forward repeatedly. Observe the HTTP request packet travelling from PC0 through the switch, through the router, to the server, and the response returning.
28. Click on an HTTP packet envelope > Outbound PDU Details. Verify Destination Port = 80.

Task 1b – HTTPS Implementation

29. In PC0's Web Browser, type: https://192.168.2.2 (note the 's').
30. In Simulation Mode, filter for HTTPS. Click Capture/Forward.

31. Click on the HTTPS packet > Outbound PDU Details. Verify Destination Port = 443.
32. Notice: In the PDU Details for HTTPS, the DATA field is encrypted – you cannot read the content. This is the key visual difference from HTTP.

Task 2 – DNS Configuration in Cisco Packet Tracer

Why set up DNS? So PCs can browse by name (e.g., http://fast-ai) instead of IP address. The DNS server holds the mapping.

33. On Server0, click Services > DNS. Turn DNS Service ON.
34. Add an A Record: Name = fast-ai, Address = 35.0.0.1 (or your web server's IP). Click Add.
35. Add a CNAME Record: Name = fast, Host Name = fast-ai. Click Add.
36. On each PC, set the DNS Server field to the IP of Server0 (192.168.2.2 in our topology).
37. Open PC0's Web Browser and type: http://fast-ai. The browser asks the DNS server to resolve 'fast-ai', receives back the IP, then sends an HTTP GET to that IP. The page loads.
38. Switch to Simulation Mode and filter for DNS. Watch the UDP query go from PC to DNS server, and the response come back with the resolved IP.

WHY UDP FOR DNS?	DNS uses UDP port 53 for standard queries (not TCP) because DNS requests and responses are small and the overhead of establishing a TCP connection would slow down every web page load. UDP is connectionless and fast. TCP is only used for DNS zone transfers (large data between name servers).

Wireshark – Theory and Complete Guide

What is Wireshark and Why Do Network Engineers Use It?

Wireshark is a free, open-source packet analyzer (often called a protocol analyzer or 'packet sniffer'). It captures every packet passing through a network interface and displays the full decoded content of each protocol layer. It is the most widely used network troubleshooting and learning tool in the industry.

Use cases include:

- Diagnosing why a network connection is slow or failing.
- Verifying that encryption is working (HTTPS traffic should be unreadable).
- Learning how protocols like TCP, DNS, and HTTP actually work by watching live packets.

- Cybersecurity investigation – examining malicious network traffic.
- Verifying application behavior – confirming your code sends the right packets.

The Wireshark Interface – Understanding Each Panel

Panel	What It Shows	How to Use It
Packet List (top)	One row per captured packet with timestamp, source, destination, protocol, length, info	Click a row to select it for inspection
Packet Details (middle)	Tree view of all protocol layers decoded for the selected packet	Expand each layer (Ethernet, IP, TCP, HTTP) to see fields
Packet Bytes (bottom)	Raw hexadecimal and ASCII representation of the packet bytes	Selecting a field in the details panel highlights its bytes here

Starting a Capture

39. Open Wireshark.
40. In the 'Capture' section on the home screen, select the network interface you want to monitor. Choose 'Wi-Fi' if connected wirelessly, or 'Ethernet' if connected via cable. The interface showing live traffic (wavy line graph) is your active one.
41. Double-click the interface to start capturing immediately.
42. Use your browser or run ping/tracert commands to generate traffic.
43. Click the red square (Stop) button when you have captured enough.

Display Filters – The Most Important Wireshark Skill

Wireshark may capture thousands of packets per second. Display filters let you show only the packets you care about. Type the filter in the bar at the top and press Enter.

Filter Expression	What It Shows
http	Only HTTP traffic
tcp.port == 443	Only HTTPS traffic (port 443)
dns	Only DNS queries and responses
ip.addr == 192.168.1.1	All traffic to or from 192.168.1.1
ip.src == 192.168.1.10	Only traffic sent FROM 192.168.1.10

tcp.flags.syn == 1	Only TCP SYN packets (connection initiations)
http.request.method == 'GET'	Only HTTP GET requests
frame contains 'password'	Frames containing the word 'password' in cleartext

PRO TIP	Right-click any field in the Packet Details panel and choose 'Apply as Filter > Selected' to instantly create a filter for that exact value. This is much faster than typing filters manually.

HTTP Packet Analysis in Wireshark – Step by Step

This exercise uses the intentionally vulnerable test site http://testphp.vulnweb.com/login.php which allows you to see HTTP credentials in cleartext – demonstrating exactly why HTTPS is mandatory.

44. Open Wireshark and start capturing on your active interface.
45. Open your browser and navigate to: http://testphp.vulnweb.com/login.php
46. Enter username: test and password: test. Click Login.
47. Stop the capture in Wireshark. Apply the filter: http
48. Look for a packet with 'POST' in the Info column (this is the login submission). Click it.
49. In Packet Details, expand HTML Form URL Encoded. You will see Form item: uname = 'test' and Form item: pass = 'test' in complete plaintext.

CRITICAL LESSON	You just read a username and password from an HTTP login form without any hacking tools – just Wireshark. This demonstrates why any website that handles logins, payments, or private data MUST use HTTPS. The same capture on an HTTPS site would show encrypted Application Data that is completely unreadable.

HTTP vs. HTTPS Comparison in Wireshark

Observation	HTTP Capture	HTTPS Capture
Destination Port	80 (visible in TCP header)	443 (visible in TCP header)
Protocol label	HTTP	TLSv1.2 or TLSv1.3
Request URL	Visible in plain text	Encrypted – not visible
Form data (passwords)	Fully readable	Encrypted – unreadable

Response body	HTML source visible	Encrypted blob
Certificate handshake	None	TLS ClientHello/ServerHello visible before encryption

Wireshark Lab Exercise – HTTPS Observation

50. Start a new Wireshark capture.
51. In your browser, navigate to https://www.google.com (or any HTTPS site).
52. Stop capture and apply filter: tcp.port == 443
53. Observe the TLS Handshake: you will see ClientHello (client initiates), ServerHello (server responds), Certificate (server sends its certificate), then Application Data (the encrypted HTTP traffic).
54. Click on a 'Application Data' packet. In Packet Details, note that the data field shows only an encrypted payload – no readable content.
55. Take screenshots of both the HTTP login analysis (readable password) and the HTTPS capture (unreadable data) for comparison.

One-line answer for the lab: HTTP transmits all data in plaintext including passwords making it completely insecure, while HTTPS encrypts all communication using TLS so even if captured the content is unreadable.

Lab 06 – DHCP, FTP, and SMTP Services

Theory: Application Layer Services

DHCP – Why Dynamic Addressing Matters

Manually assigning IP addresses to every device is practical in a lab with 5 PCs but impossible in an enterprise with 5,000 devices. DHCP (Dynamic Host Configuration Protocol) automates this. A DHCP server maintains a pool (range) of IP addresses and assigns one to each device that requests it. Along with the IP, it sends the subnet mask, default gateway, and DNS server address – everything a device needs to communicate.

The DHCP process follows the DORA sequence:

56. Discover – Client broadcasts: 'Is there a DHCP server? I need an IP.'
57. Offer – DHCP server responds: 'I offer you 192.168.1.50 for 24 hours.'
58. Request – Client broadcasts: 'I accept 192.168.1.50 from that specific server.'
59. Acknowledge – Server confirms: 'It is yours. Lease expires in 24 hours.'

WHY BROADCAST?	The Discover and Request are broadcasts because at that point the client has no IP address and does not know the DHCP server's IP. Broadcasting ensures every device on the LAN hears it. Only DHCP servers respond.

FTP – File Transfer Protocol

FTP (File Transfer Protocol) allows files to be transferred between a client and a server over a network. It uses two connections: port 21 for control commands (login, list files, change directory) and port 20 for actual data transfer. FTP sends credentials in plaintext – like HTTP, it is insecure. SFTP (SSH File Transfer Protocol) is the modern secure alternative.

In Cisco Packet Tracer, the FTP server lets you create user accounts with usernames, passwords, and file permissions (read, write, delete, rename, list).

SMTP – Email Protocol

SMTP (Simple Mail Transfer Protocol) on port 25 handles the sending of email from client to server and between mail servers. When you send an email, your email client uses SMTP to hand the message to your mail server, which uses SMTP to forward it toward the recipient's mail server. For receiving email, POP3 (port 110) or IMAP (port 143) are used.

Lab Task 1 – DHCP Server Configuration in Packet Tracer

60. Build a topology with: 1 Router (1841), 1 Switch (2960), 1 Server (as DHCP server), and 3+ PCs.
61. Assign a static IP to the server (the DHCP server itself must have a static IP – it cannot get its IP from DHCP): e.g., 192.168.1.1/24.
62. Click the Server > Services > DHCP. Turn the DHCP service ON.
63. Configure the pool: Pool Name = LAN_POOL, Default Gateway = 192.168.1.1, DNS Server = 192.168.1.1, Start IP = 192.168.1.10, Subnet Mask = 255.255.255.0, Max Users = 50. Click Save.
64. On each PC: Desktop > IP Configuration. Select DHCP. Within a few seconds the PC should receive an IP from the pool (192.168.1.10, .11, .12, etc.).
65. Why does the DHCP server have a static IP? If the DHCP server itself got its IP from DHCP, it would need a DHCP server to get started – a circular dependency. All infrastructure servers (DHCP, DNS, web) must have static IPs so they are always reachable at the same address.

WHY STATIC IPs FOR SERVERS?	If a web server's IP changed every day (because DHCP assigned a new one), no DNS record would stay accurate and clients could never reliably connect. Servers get static IPs so their address never changes. DHCP is for end-user devices (PCs, laptops, phones) that come and go.

Lab Task 1c – FTP Server Setup

66. Click Server2 (or Server3) > Services > FTP. Turn FTP ON.
67. Create an account: Username = [your first name], Password = [your roll number], File permissions: check Read, Write, Delete, Rename, List. Click Add.
68. Create a text file with name = [your last name].txt. Use the server's text editor (Services > FTP > New File).
69. From PC-A's Command Prompt: ftp 192.168.x.x (server's IP). Enter your credentials. Type dir to list files. Type get lastname.txt to download. Type put anotherfile.txt to upload.

```
PC> ftp 192.168.1.3
Connected to 192.168.1.3.
220 Welcome to the FTP server
Username: firstname
```

```
Password: rollnumber
230 Login successful.

ftp> dir
  lastname.txt   1024 bytes

ftp> get lastname.txt
226 Transfer complete.

ftp> quit
```

Lab Task 1a & 1b – SMTP Email Configuration

70. Click Server2 > Services > EMAIL (SMTP). Turn SMTP and POP3 ON. Set Domain Name = server2.local. Create account: [lastname + last3digits of roll number] with a password.
71. Click Server1 > Services > EMAIL. Turn on and create account for PC-B.
72. On PC-A: Desktop > Email. Configure: Your Name, Email Address = yourname@server2.local, Incoming Mail Server = Server2's IP, Outgoing Mail Server = Server2's IP, Username and Password.
73. Click Compose. Address it to the PC-B account on Server1. Send. Packet Tracer will simulate the SMTP transfer.
74. On PC-B: Open Email > Receive. The message should appear.

WHY TWO MAIL SERVERS?	In real email, your 'send' server (SMTP) and 'receive' server (POP3/IMAP) can be different machines. PC-A sends via Server2's SMTP, Server2 routes the mail to Server1 via SMTP server-to-server, and PC-B retrieves it from Server1 via POP3. This mirrors how Gmail, Outlook, and all real email systems work.

Lab 07 – Access Control Lists (ACLs)

Theory: What Are ACLs and Why Do We Need Them?

An Access Control List (ACL) is a set of rules applied to a router interface that permits or denies traffic based on criteria like source IP, destination IP, protocol, or port number. Without ACLs, every device on a routed network can communicate with every other device – a significant security risk in organizational networks.

ACLs implement network segmentation policies such as: 'Students cannot initiate connections to the Teachers network, but Teachers can freely access Student resources'. This is analogous to a one-way door – teachers can walk through in both directions, students can only come through one way.

Standard vs. Extended ACLs

Type	Filters Based On	Placement Rule	Use Case
Standard ACL (1-99)	Source IP address only	As close to destination as possible	Block all traffic from a subnet
Extended ACL (100-199)	Source IP, Destination IP, Protocol, Port	As close to source as possible	Allow only HTTP from specific host, block FTP etc.

ACLs have an implicit 'deny all' at the end. If a packet does not match any rule, it is dropped. This means you must explicitly permit traffic you want to allow if you have any deny rules.

Standard ACL – Direction Matters

An ACL is applied to a router interface in a specific direction: inbound (traffic arriving on the interface) or outbound (traffic leaving via the interface). For the Students-Teachers scenario, placing a standard ACL outbound on the router interface facing the Teachers network effectively blocks all Student traffic from reaching Teachers.

WHY DIRECTION MATTERS?	If you apply the deny on the inbound interface (toward students), the router processes the ACL for ALL traffic coming from the student side, including traffic destined for other places. Apply outbound on the

	Teachers interface to specifically block only traffic headed to the Teachers network while leaving other student traffic unaffected.

Lab Exercise III – Students vs. Teachers ACL

Topology Understanding

The topology has two networks: STUDENTS (192.168.1.0/24) on the left with PC2 and PC3, and TEACHERS (192.168.2.0/24) on the right with PC4 and PC5. They are connected through Router2 and Router3 (via a 192.168.3.0/24 link between the routers).

Task 1 – Block Students from reaching Teachers (one-way block)

We want Students to NOT reach Teachers, but Teachers CAN reach Students. Apply a Standard ACL on Router3's interface facing the Teachers network (outbound), denying the student subnet.

```
Router3> enable
Router3# configure terminal

! Create ACL number 10
Router3(config)# access-list 10 deny 192.168.1.0 0.0.0.255   ! deny student subnet
Router3(config)# access-list 10 permit any                    ! permit everything else

! Apply ACL outbound on the interface facing the Teachers switch
Router3(config)# interface fastEthernet 0/1
Router3(config-if)# ip access-group 10 out
Router3(config-if)# end
Router3# write memory
```

The wildcard mask 0.0.0.255 means: match any address where the first three octets are 192.168.1 (the zeros must match exactly, the 255 means 'ignore this octet'). This is the inverse of the subnet mask.

Verification:

- Ping from PC2 (Student) to PC4 (Teacher) – should FAIL (Request timed out).
- Ping from PC4 (Teacher) to PC2 (Student) – should SUCCEED.

Task 2 – Allow Only One Student (Class Rep)

Now we want to block all students EXCEPT one specific PC (e.g., PC2 = 192.168.1.2, the CR). This requires a more precise ACL: first permit the specific PC, then deny the rest of the subnet, then permit everything else.

```
! First remove the old ACL from the interface
Router3(config)# interface fastEthernet 0/1
Router3(config-if)# no ip access-group 10 out
Router3(config-if)# exit

! Delete old ACL
Router3(config)# no access-list 10

! Create new ACL 10 with CR exception
Router3(config)# access-list 10 permit 192.168.1.2        ! Allow CR (PC2)
Router3(config)# access-list 10 deny   192.168.1.0 0.0.0.255  ! Block all other students
Router3(config)# access-list 10 permit any              ! Allow Teachers and others

! Re-apply
Router3(config)# interface fastEthernet 0/1
Router3(config-if)# ip access-group 10 out
Router3(config-if)# end
Router3# write memory
```

ORDER MATTERS	ACL rules are processed top to bottom, first match wins. If you put the 'deny 192.168.1.0' rule before 'permit 192.168.1.2', the CR's packets would be denied before ever reaching the permit rule. Always put specific rules before general ones.

Verification:

- Ping from PC2 (192.168.1.2 – CR) to PC4 (Teacher) – should SUCCEED.
- Ping from PC3 (192.168.1.3 – other student) to PC4 – should FAIL.
- Ping from PC4 (Teacher) to PC3 (Student) – should SUCCEED.

To view the ACL and its hit counters (how many packets matched each rule):

```
Router3# show access-lists

Standard IP access list 10
    10 permit 192.168.1.2 (5 matches)
    20 deny   192.168.1.0  0.0.0.255 (3 matches)
    30 permit any (12 matches)
```

Appendix – Quick Reference

Common IP Address Ranges

Range	Class	Default Subnet Mask	Typical Use
10.0.0.0 – 10.255.255.255	Class A Private	255.0.0.0 (/8)	Large enterprise networks
172.16.0.0 – 172.31.255.255	Class B Private	255.255.0.0 (/16)	Medium networks
192.168.0.0 – 192.168.255.255	Class C Private	255.255.255.0 (/24)	Home/small office – most common in labs
127.0.0.1	Loopback	N/A	Testing – routes back to own machine
169.254.x.x	APIPA	255.255.0.0	Auto-assigned when DHCP fails – indicates problem

Common Port Numbers

Port	Protocol	Service	Transport
20/21	FTP	File Transfer Protocol	TCP
22	SSH	Secure Shell	TCP
23	Telnet	Remote login (unencrypted)	TCP
25	SMTP	Email sending	TCP
53	DNS	Domain Name System	UDP (queries) / TCP (zone transfers)
67/68	DHCP	Dynamic IP assignment	UDP
80	HTTP	Web (unencrypted)	TCP
110	POP3	Email receiving	TCP
443	HTTPS	Web (encrypted)	TCP
143	IMAP	Email (with server sync)	TCP

Wireshark Filter Cheat Sheet

Goal	Filter
Show only HTTP	http
Show only HTTPS	ssl or tls
Show only DNS	dns
Show only TCP	tcp
All traffic from one IP	ip.src == 192.168.1.10
All traffic to one IP	ip.dst == 192.168.1.1
Traffic between two IPs	ip.addr == 192.168.1.10 && ip.addr == 192.168.1.1
Only SYN packets (new connections)	tcp.flags.syn == 1 && tcp.flags.ack == 0
HTTP GET requests only	http.request.method == 'GET'
HTTP POST requests (form submissions)	http.request.method == 'POST'
Packets containing a keyword	frame contains 'password'
ICMP (ping traffic)	icmp
ARP traffic	arp
DHCP traffic	bootp

End of Guide – Good luck with your labs!